ENGINEERING & ARCHITECTURE DIVISION

# Software Architecture Document

## SDLC Automation Copilot

| | |
|---|---|
| **Document Type** | Software Architecture Document (SAD) |
| **System** | SDLC Automation Copilot |
| **Version** | 1.0 |
| **Date** | March 23, 2026 |
| **Organisation** | Engineering & Architecture Division |
| **Classification** | Internal Restricted |
| **Authored By** | Architecture Team |
| **Reviewed By** | Engineering Lead |

*Internal Restricted | Engineering & Architecture Division*

# Revision History

| Date | Version | Description | Author |
|---|---|---|---|
| 2026-03-23 | 1.0 | Initial SAD: significant use-cases identified, N-tier design defined, Groq + ChromaDB stack | Architecture Team |
| | | | |

# TABLE OF CONTENTS

**Revision History** **2**

**TABLE OF CONTENTS** **3**

**1. Introduction** **6**

1.1 Purpose 6

1.2 Scope 6

1.3 Definitions, Acronyms and Abbreviations 6

1.4 References 7

1.5 Document Structure 7

**2. Architectural Representation** **9**

**3. Architectural Goals and Constraints** **10**

3.1 Technical Platform 10

3.2 Transaction Management 10

3.3 Security 10

3.4 Persistence 11

3.5 Reliability and Availability 11

3.6 Performance Targets 11

3.7 Internationalisation (i18n) 11

**4. UI Specification — Screens, Buttons, Fields, Validations and Restrictions** **12**

4.1 Login Screen 12

4.1.1 Screen Description 12

4.1.2 UI Elements 12

4.1.3 Field Validations and Limitations 12

4.2 Role Selector 13

4.2.1 Screen Description 13

4.2.2 UI Elements 13

4.2.3 Conditions and Restrictions 13

4.3 Document Upload Panel 13

4.3.1 Screen Description 13

4.3.2 UI Elements 14

4.3.3 File Validations and Limitations 14

4.3.4 Functional Conditions 15

4.4 Chat Window 15
4.4.1 Screen Description 15
4.4.2 UI Elements 15
4.4.3 Message Input Validations 16
4.4.4 Functional Conditions 17
4.5 Session History Sidebar 17
4.5.1 Screen Description 17
4.5.2 UI Elements 17
4.5.3 Validations and Conditions 18
**5. User Roles, Permissions, and Functionality Mapping 19**
5.1 Role Descriptions 19
5.2 Role × Functionality Matrix 19
5.3 Role-Specific Functional Conditions 20
**6. Data Flow 21**
6.1 Document Ingestion Flow 21
6.1.1 Step-by-Step Flow 21
6.1.2 Role-Specific Checks in Ingestion 21
6.2 RAG Query Generation Flow 22
6.2.1 Step-by-Step Flow 22
6.2.2 Role-Specific Checks in Query Flow 23
**7. High-Level Architecture 24**
7.1 Tier 1 — Client Tier (Next.js 16 Frontend) 24
7.2 Tier 2 — Application Tier (FastAPI + RAG Service) 24
7.3 Tier 3 — Intelligence Tier (Ollama LLM + Embedding Model) 24
7.4 Tier 4 — Data Tier (Qdrant + SQLite + File Storage) 25
7.5 Cross-Tier Communication Summary 25
**8. Component Architecture — Open-Source Selection, Rationale and Comparison 27**
8.1 LLM Inference — Ollama with Mistral-7B / LLaMA-3-8B 27
8.2 Vector Database — Qdrant 27
8.3 Embedding Model — sentence-transformers/all-MiniLM-L6-v2 28
8.4 Backend Framework — FastAPI 29
8.5 RAG Orchestration — LangChain 30
8.6 Frontend Framework — Next.js 16 (React 19) 30
8.7 Model Training — Hugging Face PEFT (LoRA) + MLflow 31

**9. Model Training and Continuous Improvement Flow 32**

9.1 Training Strategy Overview 32

9.2 Step 1 — Feedback Collection 32

9.3 Step 2 — Dataset Curation 32

9.4 Step 3 — LoRA Fine-Tuning 33

9.5 Step 4 — Evaluation 33

9.6 Step 5 — Adapter Deployment 33

9.7 MLflow Experiment Tracking — Logged Artefacts 34

**10. Logical View 35**

10.1 N-Tier Layer Overview 35

10.2 Primary Functional Areas 35

10.3 Use-Case Realizations 35

**11. Process View 37**

11.1 Session State Machine 37

The session lifecycle state machine governs how a user session transitions through operational states from creation through document generation: 37

11.2 Concurrency Model 37

**12. Deployment View 38**

12.1 On-Premise Deployment Topology 38

12.2 Future Update Roadmap — Containerisation and Scale-Out 38

**13. Implementation View 39**

13.1 Monorepo Structure 39

13.2 Key Layers and Modules 39

**14. Data View 41**

14.1 Relational Database — SQLite 41

14.2 Vector Database — Qdrant Collection Schema 42

**15. Size and Performance 43**

15.1 Volume Estimates 43

15.2 Performance Targets 43

**16. Quality Attributes 44**

# 1. Introduction

This document is the Software Architecture Document (SAD) for the SDLC Automation Copilot — a fully on-premise, open-source AI-powered workspace that automates generation of software development lifecycle artefacts (BRDs, FRDs, Test Packs) from uploaded reference documents. This version replaces all third-party API dependencies with open-source, self-hosted components and extends the document with full UI specifications, role-function mappings, data flow descriptions per role, component selection rationale, and a model training pipeline.

## 1.1 Purpose

The SAD provides a comprehensive architectural overview using the RUP '4+1' view model [KRU41]. It is intended to capture every significant architectural decision and provide sufficient detail for designers, developers, data architects, DevOps engineers, and business stakeholders to implement, operate, and extend the system without external documentation.

## 1.2 Scope

In scope for :

- AI-powered RAG pipeline: document ingestion, chunking, embedding, and vector retrieval — entirely on-premise.
- Next.js 16 frontend: role-based UI with full field-level specification, validations, and restrictions.
- FastAPI backend: orchestration, REST/SSE endpoints, SQLite session management.
- Qdrant (on-premise Docker) as the production vector store.
- Ollama serving Mistral-7B / LLaMA-3-8B as the on-premise LLM inference engine.
- sentence-transformers/all-MiniLM-L6-v2 as the local embedding model.
- MLflow + LoRA fine-tuning pipeline for continuous model improvement.
- Complete role × functionality matrix, UI specification, and component comparison tables.

Out of scope for : CI/CD automation pipelines, OAuth2 external identity providers, Jira synchronisation, cloud-hosted LLM inference, multi-tenant architecture.

## 1.3 Definitions, Acronyms and Abbreviations

| Term | Definition |
|---|---|
| SAD | Software Architecture Document |
| RAG | Retrieval-Augmented Generation — vector search combined with LLM generation to ground responses in uploaded source documents |
| BRD | Business Requirements Document |
| FRD | Functional Requirements Document |
| BA | Business Analyst |
| FBA | Functional Business Analyst |
| QA | Quality Assurance Engineer |
| LLM | Large Language Model |
| ASGI | Asynchronous Server Gateway Interface |
| ORM | Object-Relational Mapper — SQLAlchemy in this system |
| SSE | Server-Sent Events — used for live markdown token streaming from backend to frontend |

| Term | Definition |
|---|---|
| JWT | JSON Web Token — used for session authentication |
| TTFT | Time-To-First-Token — LLM streaming latency metric |
| LoRA | Low-Rank Adaptation — parameter-efficient LLM fine-tuning technique |
| PEFT | Parameter-Efficient Fine-Tuning — Hugging Face library for LoRA training |
| HNSW | Hierarchical Navigable Small World — approximate nearest-neighbour index algorithm used by Qdrant |
| MLflow | Open-source ML lifecycle and experiment tracking platform |
| Ollama | Open-source LLM runtime for local model execution |
| RUP | Rational Unified Process |
| UML | Unified Modelling Language |

## 1.4 References


- [KRU41] The '4+1' View Model of Software Architecture — Philippe Kruchten, IEEE Software, Nov 1995.
- [LANGCHAIN] LangChain Documentation — https://docs.langchain.com
- [QDRANT] Qdrant Documentation — https://qdrant.tech/documentation
- [OLLAMA] Ollama Documentation — https://ollama.com/docs
- [MISTRAL] Mistral-7B Model Card — https://huggingface.co/mistralai/Mistral-7B-v0.1
- [LLAMA3] LLaMA-3-8B Model Card — https://huggingface.co/meta-llama/Meta-Llama-3-8B
- [FASTAPI] FastAPI Documentation — https://fastapi.tiangolo.com
- [NEXTJS] Next.js 16 Documentation — https://nextjs.org/docs
- [HF] HuggingFace sentence-transformers — https://huggingface.co/sentence-transformers
- [MLFLOW] MLflow Documentation — https://mlflow.org/docs
- [PEFT] Hugging Face PEFT / LoRA — https://huggingface.co/docs/peft


## 1.5 Document Structure

| Section | Content |
|---|---|
| Section 2 | Architectural representation — views and audiences |
| Section 3 | Architectural goals and constraints — platform, security, performance |
| Section 4 | UI Specification — all screens, buttons, fields, validations, limitations |
| Section 5 | User roles, permissions, and role × functionality matrix |
| Section 6 | Data Flow — ingestion and RAG query flow with role-specific checks |
| Section 7 | High-Level Architecture — all components and flows described in detail |
| Section 8 | Component Architecture — open-source selection rationale and comparison tables |
| Section 9 | Model Training Flow — fine-tuning pipeline and continuous improvement strategy |
| Section 10 | Logical View — N-tier layers and design packages |
| Section 11 | Process View — session state machine and concurrency model |
| Section 12 | Deployment View — on-premise topology and port mapping |

| Section | Content |
|---|---|
| Section 13 | Implementation View — codebase structure and layer descriptions |
| Section 14 | Data View — relational schema (SQLite) and vector schema (Qdrant) |
| Section 15 | Size and Performance — volume estimates and performance targets |
| Section 16 | Quality Attributes — scalability, reliability, security, maintainability |

# 2. Architectural Representation

This SAD uses the RUP '4+1' view model [KRU41] supplemented with a Data View and a dedicated UI Specification section to satisfy stakeholder requirements across all disciplines.

| View | Primary Audience | Content |
|---|---|---|
| Use-Case View | BAs, FBAs, QAs, Product Owners | Significant scenarios; role × functionality matrix; UI conditions |
| Logical View | Designers, Senior Architects | N-tier layer diagram, RAG pipeline, design packages |
| Process View | Integrators, Backend Engineers | Session state machine, async ASGI event loop, SSE concurrency |
| Deployment View | DevOps, Infrastructure | On-premise topology, port mapping, Docker containers |
| Implementation View | Developers | Directory structure, module descriptions, technology choices |
| Data View | Data Architects, DBAs | SQLite ER diagram, Qdrant collection schema |
| UI Specification | Frontend Developers, UX Designers | All screens, buttons, input fields, validations, restrictions |

# 3. Architectural Goals and Constraints

## 3.1 Technical Platform

All components are open-source, self-hosted, and on-premise. Zero external API dependencies exist in.

| Component | Technology | Notes |
|---|---|---|
| Frontend Runtime | Node.js 18+ / Next.js 16 (React 19) | File-based routing; SSE-compatible ReadableStream |
| Backend Runtime | Python 3.10+ / FastAPI + Uvicorn (ASGI) | Native async; non-blocking LLM calls; auto OpenAPI docs |
| LLM Inference | Ollama (Mistral-7B or LLaMA-3-8B) — on-premise | GPU-accelerated; zero API cost; no external dependency |
| Embedding Model | sentence-transformers/all-MiniLM-L6-v2 (local) | 384-dim vectors; loaded in-process; zero API cost |
| Vector Store | Qdrant (Docker, on-premise) | HNSW indexing; metadata filtering; REST + gRPC |
| Relational DB | SQLite via SQLAlchemy ORM | Zero-config ; migration path to PostgreSQL in future updates |
| Model Training | Hugging Face Trainer + PEFT (LoRA) + MLflow | Experiment tracking; parameter-efficient fine-tuning |
| Virus Scanning | ClamAV (open-source) | On-premise file scanning before ingestion |
| OS (Production) | Linux Ubuntu 22.04 LTS | CUDA-compatible; systemd process management |

## 3.2 Transaction Management

- Session creation and message persistence are wrapped in SQLAlchemy database transactions with rollback support.
- The RAG retrieval and LLM generation pipeline is stateless between calls — no distributed transaction manager is required.
- The streaming endpoint (/api/chat/query/stream) commits the session record atomically before streaming begins.
- Document ingestion (parse, chunk, embed, persist to Qdrant) is treated as an idempotent batch operation.

## 3.3 Security

- Authentication: Username and password login with bcrypt password hashing. JWT tokens issued on login (RS256 in ).
- Authorisation: Role-based context injection (BA, FBA, QA) restricts the prompt scope per user role.
- Transport: All client-to-server traffic encrypted via HTTPS/TLS through the Nginx reverse proxy.
- Audit Trail: Every message persisted to SQLite with timestamp, session ID, and role — immutable once written.
- Virus Scanning: ClamAV scans all uploaded files before ingestion; infected files rejected with error.
- No External APIs: No credentials or API keys are transmitted to external services.

## 3.4 Persistence

- Relational (SQLite): User accounts, chat sessions, message history, document metadata, feedback records.
- Vector (Qdrant): 384-dimensional chunk embeddings with metadata, persisted to Docker volume on disk.
- File Storage: Uploaded source documents stored on local filesystem or NAS; path referenced in SQLite.

## 3.5 Reliability and Availability

- Target: 99% uptime during business hours (08:00–22:00).
- Stateless Backend: FastAPI restarts without data loss — all state externalised.
- Ollama: Fully on-premise; supervised via systemd; eliminates external LLM availability risk.
- Retry Logic: Exponential backoff on Ollama inference timeouts; errors surfaced via SSE stream.

## 3.6 Performance Targets

| Metric | Target |
|---|---|
| Time-To-First-Token (TTFT) | < 3 seconds at standard load with GPU-accelerated Ollama |
| Qdrant cosine similarity search (top-k = 5) | < 500 ms for corpus up to 50,000 chunks |
| Concurrent streaming sessions (no degradation) | ≥ 10 simultaneous users |
| Full document ingestion — 20-page PDF | < 60 seconds (parse + chunk + embed + store) |
| UI page load time | < 2 seconds on local network |
| Login response time | < 500 ms |

## 3.7 Internationalisation (i18n)

This version targets English-language documents and UI exclusively. Provisions for future i18n: the prompt template layer is locale-agnostic, Next.js is compatible with next-i18next, and all-MiniLM-L6-v2 supports multilingual embeddings architecturally.

# 4. UI Specification — Screens, Buttons, Fields, Validations and Restrictions

This section defines every user-facing screen in the SDLC Automation Copilot. For each screen all interactive elements (buttons, input fields, labels, icons) are described along with their validation rules, character limits, file restrictions, and functional conditions governing use.

## 4.1 Login Screen

### 4.1.1 Screen Description

The Login Screen is the application entry point. It presents a centred card on a neutral background containing the application logo, a username field, a password field, and a Login button. No other navigation is accessible before authentication.

### 4.1.2 UI Elements

| Element | Type | Label / Placeholder | Action / Behaviour |
|---|---|---|---|
| Application Logo | Static image | SDLC Automation Copilot | Decorative; no action |
| Username Field | Text input | 'Username' / 'Enter your username' | Standard text; whitespace auto-trimmed on submit |
| Password Field | Password input | 'Password' / 'Enter your password' | Characters masked; show/hide toggle (eye icon) on right |
| Show/Hide Toggle | Icon button (eye) | Tooltip: 'Show password' | Toggles input type between password and text |
| Login Button | Primary button | 'Login' | Submits credentials via POST /api/auth/login; disabled while request is in-flight |
| Error Banner | Alert (red border) | 'Invalid username or password. Please try again.' | Appears below fields on HTTP 401; auto-dismisses after 5 seconds |
| Lockout Banner | Alert (red border) | 'Too many failed attempts. Account locked for 10 minutes.' | Appears after 5 consecutive failures within 10 minutes |

### 4.1.3 Field Validations and Limitations

| Field | Rule | Error Message Shown Inline |
|---|---|---|
| Username | Required. Length: 3–50 characters. Pattern: alphanumeric and underscore only (a-z, A-Z, 0-9, _). No spaces. | 'Username must be 3–50 characters (letters, numbers, underscore only)' |
| Password | Required. Length: 8–128 characters. No leading or trailing whitespace. | 'Password must be 8–128 characters' |
| Both fields blank | Both fields highlighted red on submit attempt. | 'Please enter your username and password' |
| Rate limit | Maximum 5 failed attempts within 10 minutes triggers account lock. | 'Too many attempts. Account locked for 10 minutes.' |

| Field | Rule | Error Message Shown Inline |
|---|---|---|
| Session token | JWT returned on success; stored in HTTP-only cookie (not localStorage). | N/A |

## 4.2 Role Selector

### 4.2.1 Screen Description

Displayed immediately after successful login and before any session can be created. A modal overlay with three role cards (BA, FBA, QA) is presented. The user must select a role and confirm before accessing the chat interface. The selected role persists for the entire session and is displayed in the application header.

### 4.2.2 UI Elements

| Element | Type | Label / Content | Action / Behaviour |
|---|---|---|---|
| Role Card — BA | Selectable card | 'Business Analyst' / 'Generate BRDs, business glossaries, and stakeholder analyses' | Highlights with blue border on selection; mutually exclusive with other role cards |
| Role Card — FBA | Selectable card | 'Functional Business Analyst' / 'Generate FRDs, API specs, and data flow documents' | Highlights with blue border on selection |
| Role Card — QA | Selectable card | 'Quality Assurance Engineer' / 'Generate Test Plans, Test Packs, and traceability matrices' | Highlights with blue border on selection |
| Start Session Button | Primary button | 'Start Session' | Disabled until a role card is selected; creates new session on click via POST /api/chat/sessions; navigates to chat window |
| Role Description Tooltip | Info icon per card | One-sentence role description on hover | Static tooltip; no action |

### 4.2.3 Conditions and Restrictions

- A role must be selected before any session can be created. The chat interface and all upload controls are hidden until role confirmation.
- Role cannot be changed mid-session. To switch roles the user must open a new session via the '+ New Session' button in the sidebar.
- The selected role is stored in the Sessions table (SQLite) and injected into every LLM prompt for the duration of that session.
- The role badge is displayed at all times in the application header once confirmed.

## 4.3 Document Upload Panel

### 4.3.1 Screen Description

Located in the left sidebar above the session history list. Allows users to upload reference documents that are ingested into the Qdrant vector store for RAG retrieval. The panel shows upload progress, file status badges, and error states.

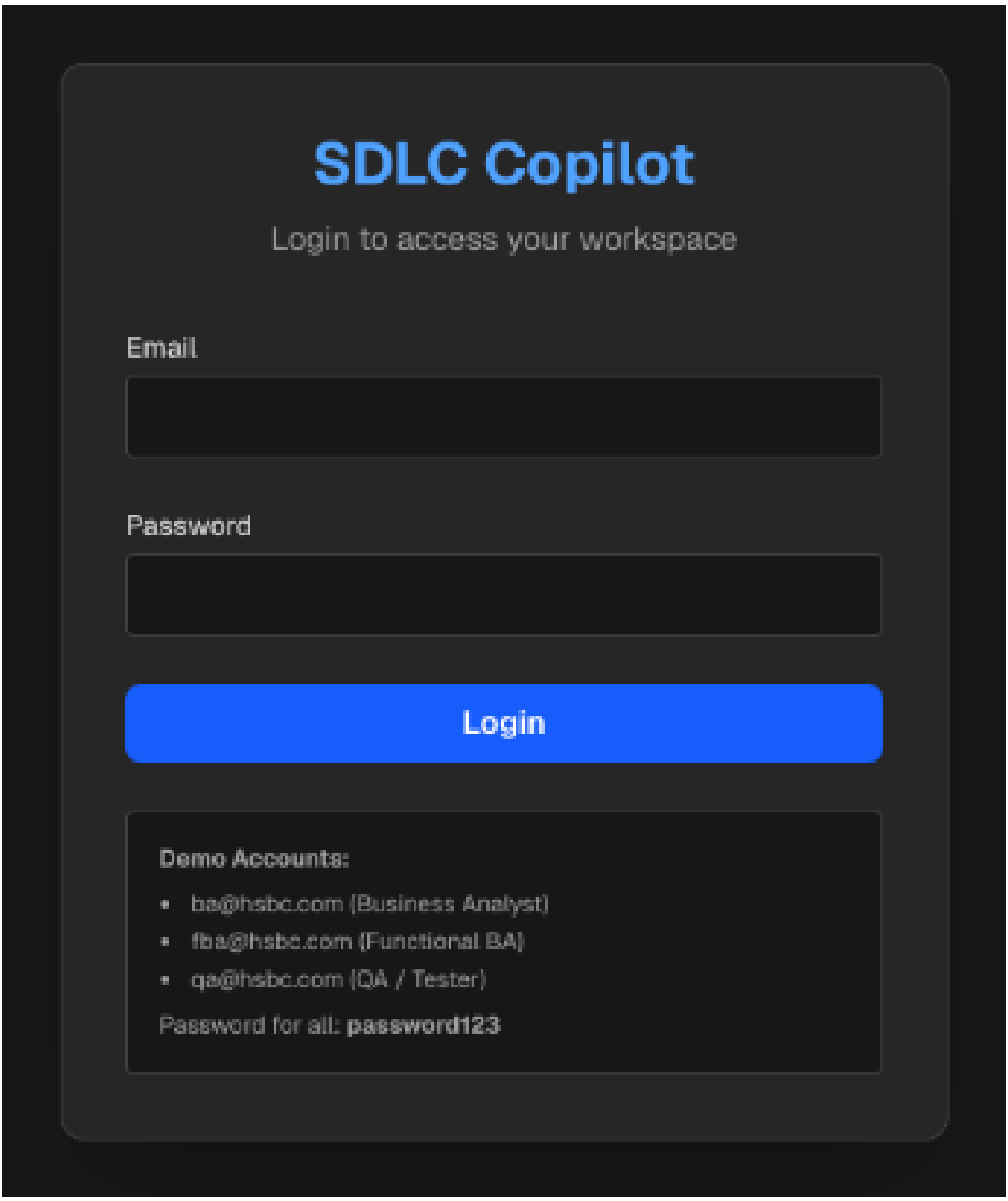


*Figure 1 — Login Interface (UI Screenshot)*

### 4.3.2 UI Elements

| Element | Type | Label / Placeholder | Action / Behaviour |
|---|---|---|---|
| Upload Drop Zone | Drag-and-drop area + 'Browse Files' link | 'Drop PDF, DOCX or CSV files here, or click to browse' | Opens OS file picker on click; accepts drag-and-drop; highlights border on drag-over |
| File Picker (hidden input) | File input (accept=.pdf,.docx,.csv) | — | Triggered by Browse Files link; multi-file selection supported |
| File List Item | List row per file | Filename + file size + status badge | Populated after file selection; updates in real time |
| Status Badge | Colour label per file | Pending (grey) / Processing (blue spinner) / Ready (green) / Error (red) | Updates via SSE status channel from backend |
| Priority Tag Dropdown | Single-select dropdown (QA role only) | 'Priority: High / Medium / Low' | Visible only for QA role; defaults to 'Medium'; stored in chunk metadata |
| Upload & Process Button | Primary button | 'Upload & Process' | Triggers POST /api/documents/upload; disabled if no files are selected or all files already uploaded |
| Remove File Icon | Icon button (trash, per file) | Tooltip: 'Remove' | Removes file from pending queue before upload; disabled after status = Processing or Ready |
| Progress Bar | Linear bar per file | 'Processing chunk X of Y…' | Shown during ingestion; hidden when status = Ready or Error |
| Error Tooltip | Red icon on Error badge | '[Error message from server]' | Hover to see reason (e.g. 'File too large', 'Unsupported format', 'Virus detected') |
| Clear Completed Button | Secondary button | 'Clear Completed' | Removes all Ready-status entries from display list (does not remove from Qdrant) |

### 4.3.3 File Validations and Limitations

| Rule | Limit / Constraint | Error Message or Behaviour |
|---|---|---|
| Accepted formats | PDF (.pdf), DOCX (.docx), CSV (.csv) only | Toast notification: 'Unsupported file type. Only PDF, DOCX and CSV are accepted.' |
| Maximum file size | 25 MB per file | Toast: 'File exceeds 25 MB limit. Please reduce file size.' |
| Maximum files per session | 10 files | Toast: 'Maximum 10 files per session. Remove a file to add another.' |
| Maximum total upload per session | 100 MB cumulative | Toast: 'Total upload limit of 100 MB reached for this session.' |

| Rule | Limit / Constraint | Error Message or Behaviour |
|---|---|---|
| Minimum file size | 1 KB (empty files rejected) | Toast: 'File appears to be empty and cannot be processed.' |
| Filename length | Maximum 200 characters | Filenames truncated to 200 characters; special characters (< > : " / \ \| ? *) replaced with underscore |
| Duplicate detection | Same filename + same file size within same session | Warning dialog: 'This file appears to already be uploaded. Proceed anyway?' with Continue / Cancel buttons |
| Virus scan | ClamAV scan before ingestion | Toast: 'File rejected: potential malware detected. Please contact your administrator.' |
| Password-protected PDFs | Rejected at parse stage | Status badge = Error; tooltip: 'PDF is password-protected and cannot be parsed.' |

### 4.3.4 Functional Conditions

- Files are scoped to the session in which they are uploaded. They are not shared across sessions.
- A document can only be queried after its status badge shows 'Ready'. Queries submitted while any file is in 'Processing' state display a banner: 'Documents are still being indexed. Responses may be incomplete.'
- BA and FBA roles can upload PDF, DOCX, and CSV. QA role additionally displays a Priority Tag dropdown per file.
- Already-ingested files (status = Ready) cannot be removed from the session. Only pending files can be cancelled before upload.

## 4.4 Chat Window

### 4.4.1 Screen Description

The central area of the application. Displays the message history for the active session as a vertical thread of message bubbles. User messages appear right-aligned (blue) and assistant messages left-aligned (white, markdown-rendered). The message input bar is pinned to the bottom.

### 4.4.2 UI Elements

| Element | Type | Label / Placeholder | Action / Behaviour |
|---|---|---|---|
| Message History Pane | Scrollable list | — | Auto-scrolls to bottom on new message; infinite scroll loads older messages upward |
| User Message Bubble | Right-aligned bubble (blue background) | — | Displays user query text; timestamp shown on hover |
| Assistant Message Bubble | Left-aligned bubble (white, shadow) | — | Renders streaming markdown with syntax-highlighted code blocks; timestamp shown on hover |

| Element | Type | Label / Placeholder | Action / Behaviour |
|---|---|---|---|
| Role Context Tag | Badge above assistant bubble | '[BA Context]' / '[FBA Context]' / '[QA Context]' | Indicates which role prompt was applied; non-interactive |
| Streaming Cursor | Animated blinking bar | — | Shown at the end of the assistant message while streaming; hidden on stream completion |
| Message Input Box | Multiline textarea | 'Type your query here… (Shift+Enter for new line, Enter to send)' | Auto-expands up to 6 visible lines before scrolling; supports paste |
| Character Counter | Static text (bottom-right of input) | 'X / 4000' | Turns red when > 3800 characters; submit disabled when > 4000 |
| Send Button | Icon button (paper-plane) | Tooltip: 'Send query (Enter)' | Disabled while response is streaming or input is empty/invalid; re-enabled on stream completion |
| Stop Button | Icon button (square) | Tooltip: 'Stop generation' | Visible only during streaming; aborts SSE stream on click; send button re-enabled |
| Copy Button | Icon button (copy, per assistant message) | Tooltip: 'Copy response' | Copies full assistant message text to clipboard; shows 'Copied!' toast for 2 seconds |
| Export Button | Icon button (download, per assistant message) | Tooltip: 'Export as Markdown (.md)' | Downloads the assistant response as a .md file; filename = session_title + timestamp |
| Thumbs Up / Down | Icon buttons (per assistant message) | Tooltips: 'Good response' / 'Poor response' | Submits feedback to Feedback table; thumbs-down reveals optional correction text field (max 500 chars) |
| Session Title | Editable text (header of chat window) | Auto-generated from first query (max 80 chars, truncated with ellipsis) | Click to open inline edit; saved on Enter or focus loss |
| No Documents Banner | Yellow warning banner | 'Please upload at least one reference document before sending a query.' | Shown when session has no Ready documents; hides once first document is Ready |
| Timeout Error | Red inline message | 'The model did not respond within 30 seconds. Please try again.' | Shown when Ollama inference exceeds 30-second timeout; clears on next successful query |

### 4.4.3 Message Input Validations

| Rule | Constraint | Behaviour |
|---|---|---|
| Minimum length | 3 characters | Inline hint below input: 'Query is too short (minimum 3 characters)'; Send disabled |

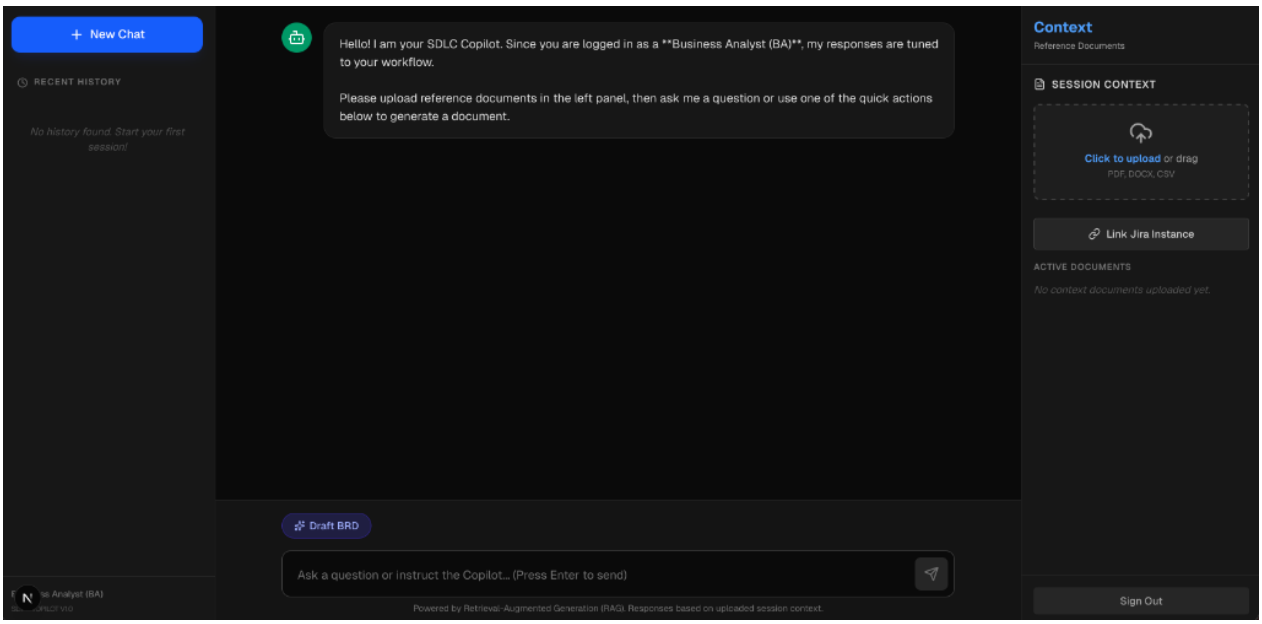


*Figure 2 — Main Workspace Dashboard (UI Screenshot)*

| Rule | Constraint | Behaviour |
|---|---|---|
| Maximum length | 4,000 characters | Character counter turns red at 3,800; submit disabled at 4,001+; tooltip: 'Query exceeds maximum length of 4,000 characters' |
| Empty / whitespace-only | Blocked | Send button remains disabled; no server call made |
| Concurrent queries | One in-flight query per session | Send button and input box disabled while a response is streaming |
| Input sanitisation | Server-side escaping of all special characters before prompt assembly | Prevents prompt injection; transparent to user |
| Session title edit | 3–80 characters; alphanumeric, spaces, hyphens, underscores | Inline error: 'Title must be 3–80 characters' |
| Feedback correction text | 0–500 characters optional | Character counter shown; submit disabled above 500 |

### 4.4.4 Functional Conditions

- If no documents are Ready in the current session, the Send button is disabled and the No Documents Banner is shown.
- Every assistant response is prefixed with a Role Context Tag ([BA Context], [FBA Context], or [QA Context]) to make the prompt framing visible.
- All messages are auto-saved to SQLite in real time. No manual save is required.
- Feedback submitted (thumbs-up or thumbs-down) is stored in the Feedback table and contributes to future LoRA fine-tuning dataset curation.
- If the Ollama inference server is unreachable, an error message is shown inline: 'The AI model is currently unavailable. Please contact your administrator.'

## 4.5 Session History Sidebar

### 4.5.1 Screen Description

The left sidebar, below the upload panel. Displays a list of all past sessions for the authenticated user, sorted by creation date descending. Allows users to resume, rename, search, and delete sessions.

### 4.5.2 UI Elements

| Element | Type | Label / Behaviour |
|---|---|---|
| Session Search Input | Text input | 'Search sessions…' — filters session list in real time (debounced 300 ms); minimum 2 characters to trigger filter |
| Session Card | Clickable list item | Displays session title (truncated at 40 chars with ellipsis), role badge (colour-coded), and creation date |
| Role Badge on Card | Colour label | BA = blue, FBA = green, QA = orange; non-interactive |
| Active Session Indicator | Bold text + blue left border on card | Highlights the currently open session |
| Rename Session | Click on title text in card | Inline edit field; 3–80 characters; saved on Enter or focus loss; cancelled on Escape |

| Element | Type | Label / Behaviour |
|---|---|---|
| Delete Session Button | Icon button (trash, appears on card hover) | Opens confirmation dialog: 'Delete this session and all its messages? This action cannot be undone.' — two buttons: 'Delete' (red) and 'Cancel' |
| New Session Button | Primary button (sidebar header) | '+ New Session' — opens Role Selector modal to create a fresh session |
| Session Count Label | Static text (sidebar header) | 'X sessions' — updates dynamically |

### 4.5.3 Validations and Conditions

- Session search: maximum 100 characters; special characters allowed (no pattern restriction for search).
- Maximum 200 active sessions per user. When the limit is reached, the oldest session with no activity in 30+ days is auto-archived. The user sees a notice: 'Session limit reached. Oldest inactive sessions are archived automatically.'
- Deleted sessions are soft-deleted (deleted_at timestamp set); permanently purged after 30 days.
- A deleted session's documents remain in Qdrant until a background cleanup job removes them (runs nightly).

# 5. User Roles, Permissions, and Functionality Mapping

The SDLC Automation Copilot implements three primary user roles. The selected role determines the LLM prompt template injected for every query, the document types the system is optimised to generate, and the UI features accessible to that role.

## 5.1 Role Descriptions

| Role | Full Name | Primary Responsibility | Documents Generated |
|---|---|---|---|
| BA | Business Analyst | Captures and articulates business needs, goals, and stakeholder requirements | Business Requirements Documents (BRD), Business Glossaries, Stakeholder Impact Analyses, Scope Statements |
| FBA | Functional Business Analyst | Translates business requirements into functional system specifications | Functional Requirements Documents (FRD), API Specifications, Data Flow Descriptions, System Integration Specs |
| QA | Quality Assurance Engineer | Designs and documents test strategies, test cases, and coverage matrices | Test Plans, Test Case Packs, Edge Case Registers, Regression Suites, Traceability Matrices |

## 5.2 Role × Functionality Matrix

Legend: ✓ = Permitted ✗ = Not Permitted ○ = Partial / Conditional

| Functionality | BA | FBA | QA |
|---|---|---|---|
| Upload PDF documents | ✓ | ✓ | ✓ |
| Upload DOCX documents | ✓ | ✓ | ✓ |
| Upload CSV documents | ✓ | ✓ | ✓ |
| Apply Priority Tag (High/Medium/Low) to uploaded files | ✗ | ✗ | ✓ |
| Generate Business Requirements Document (BRD) | ✓ | ○ (summary-level only) | ✗ |
| Generate Functional Requirements Document (FRD) | ✗ | ✓ | ○ (requirements reference only) |
| Generate Test Plan / Test Pack | ✗ | ✗ | ✓ |
| Generate API Specification | ✗ | ✓ | ✗ |
| Generate Traceability Matrix | ○ (requirements column only) | ○ (functional column only) | ✓ (full matrix) |
| Generate Business Glossary | ✓ | ○ | ✗ |
| Export assistant response as Markdown (.md) | ✓ | ✓ | ✓ |
| Copy assistant response to clipboard | ✓ | ✓ | ✓ |

| Functionality | BA | FBA | QA |
|---|---|---|---|
| Submit response feedback (thumbs up / down) | ✓ | ✓ | ✓ |
| View own session history | ✓ | ✓ | ✓ |
| View other users' sessions | ✗ | ✗ | ✗ |
| Rename own sessions | ✓ | ✓ | ✓ |
| Delete own sessions | ✓ | ✓ | ✓ |
| Stop streaming response mid-generation | ✓ | ✓ | ✓ |
| Change role mid-session | ✗ | ✗ | ✗ |
| Access admin analytics dashboard (planned future updates) | ✗ | ✗ | ✗ |

## 5.3 Role-Specific Functional Conditions

### BA Role

- Prompt template instructs the LLM to focus on: stakeholder identification, business goal articulation, problem statement framing, scope boundaries, assumptions, and constraints.
- The LLM is explicitly instructed to avoid API-level detail, technical implementation specifics, and test case language in BA outputs.
- BA users cannot trigger FRD or Test Pack generation. Attempting to request these document types produces a guidance message: 'This document type is available for FBA or QA roles. Please open a new session with the appropriate role.'

### FBA Role

- Prompt template instructs the LLM to focus on: functional requirements (shall statements), data entity definitions, process flow descriptions, system interface specifications, and validation rules.
- FBA users may reference BRD outputs from prior BA sessions as additional uploaded context documents.
- API specification generation is triggered only when uploaded documents contain structured API or integration content. If no such content is detected, the system notifies: 'No API-related content found in the uploaded documents. API specification generation may be limited.'

### QA Role

- Prompt template instructs the LLM to focus on: test objective, test scope, preconditions, test steps, expected results, edge cases, negative test scenarios, and traceability to requirements.
- Priority Tag (High / Medium / Low) is embedded in Qdrant chunk metadata. During retrieval, High-priority chunks receive a score multiplier of 1.3× in the cosine similarity ranking.
- QA-generated Test Packs always include a structured output table with columns: Test ID, Test Scenario, Preconditions, Test Steps, Expected Result, Priority, Status.

# 6. Data Flow

This section describes the end-to-end data flow for the two primary operations — document ingestion and RAG query generation — with all role-specific checks, validations, and open-source component interactions documented at each step.

## 6.1 Document Ingestion Flow

### 6.1.1 Step-by-Step Flow

1. User selects files via the Upload Drop Zone or Browse Files button and clicks 'Upload & Process'.
2. Browser (client-side validation): file type, size, count, filename length, and duplicate detection are checked. Invalid files are rejected with inline error toasts. No server call is made for rejected files.
3. Browser sends multipart/form-data POST /api/documents/upload with file payloads + session_id + role in the request body.
4. FastAPI (server-side validation): file type re-validated via MIME type inspection; size checked; ClamAV virus scan executed. Rejected files return HTTP 400 with structured error JSON.
5. FastAPI writes a Documents record to SQLite with status = 'Processing', original filename, session_id, and filesystem path (file saved to local storage).
6. FastAPI emits an SSE event on the status channel: { file_id, status: 'Processing' }. Frontend updates the file badge.
7. The RAG Ingestion Service (LangChain) is invoked asynchronously (background task):
   - Parser selection: PDF → PyPDF2; DOCX → python-docx; CSV → Pandas DataFrameLoader.
   - Chunking: LangChain RecursiveCharacterTextSplitter (chunk_size=512, chunk_overlap=50) splits text into semantically coherent segments.
   - Embedding: all-MiniLM-L6-v2 (loaded locally via sentence-transformers) converts each chunk to a 384-dimensional dense vector.
   - Metadata per chunk: { source: filename, chunk_index: N, session_id: UUID, role: BA|FBA|QA, priority_tag: High|Medium|Low }. Priority tag populated from upload panel selection (QA role only; defaults to 'Medium').
8. Chunks and embeddings are upserted into the Qdrant collection via the qdrant-client Python library.
9. SQLite Documents record status updated to 'Ready'. FastAPI emits SSE event: { file_id, status: 'Ready' }. Frontend badge updates from 'Processing' to 'Ready'.
10. On any failure (parse error, embedding error, Qdrant write error), status is set to 'Error', the error is logged, and an SSE error event is emitted. User sees the Error badge with a descriptive tooltip.

### 6.1.2 Role-Specific Checks in Ingestion

| Check | BA | FBA | QA |
|---|---|---|---|
| role field injected into chunk metadata | 'BA' | 'FBA' | 'QA' |
| priority_tag field in chunk metadata | 'Medium' (not user-configurable) | 'Medium' (not user-configurable) | User-selected: High / Medium / Low; defaults to 'Medium' |

| Check | BA | FBA | QA |
|---|---|---|---|
| BRD-oriented extraction hints in chunking | Applied — chunker biases toward goal/ stakeholder paragraphs via section-aware splitting | Not applied | Not applied |
| Test-scenario extraction hints in chunking | Not applied | Not applied | Applied — chunker biases toward acceptance criteria and edge-case paragraphs |
| Priority tag metadata written to Qdrant | Written as 'Medium' | Written as 'Medium' | Written as user-selected value |

## 6.2 RAG Query Generation Flow

### 6.2.1 Step-by-Step Flow

11. User types a query in the Message Input Box and presses Enter or clicks the Send button.
12. Browser (client-side validation): query length (3–4,000 characters) checked. Invalid input blocked with inline error; no server call made.
13. Browser sends POST /api/chat/query/stream with JSON body: { query: string, session_id: UUID, role: BA|FBA|QA }.
14. FastAPI (server-side checks): JWT token validated; session ownership verified (session must belong to authenticated user) — HTTP 403 returned if check fails; query sanitised against prompt injection patterns.
15. FastAPI calls get_context(query, session_id, role) on the RAG Service (LangChain):
    - Query embedded: all-MiniLM-L6-v2 encodes the query string into a 384-dimensional query vector (in-process, no API call).
    - Qdrant similarity search: top_k=5 chunks retrieved using cosine similarity, filtered by { session_id }. QA role applies a score multiplier of 1.3× to chunks with priority_tag='High' before final ranking.
    - Retrieved chunks concatenated into a context string (total token budget: 2,048 tokens maximum).
16. FastAPI calls assemble_prompt(context_chunks, role_template, user_query):
    - Role-specific system prompt injected: BA → BRD framing (business goals, stakeholders, scope); FBA → FRD framing (functional requirements, interfaces); QA → Test Pack framing (test cases, edge cases).
    - Final prompt structure: [System Role Prompt] + [Context Chunks] + [User Query].
17. FastAPI sends POST http://localhost:11434/api/generate to the local Ollama server with { model: 'mistral', prompt: assembled_prompt, stream: true }.
18. Ollama streams token chunks back. FastAPI async generator yields each token as an SSE event: data: { token: '...' }.
19. Browser ReadableStreamDefaultReader decodes each SSE chunk and appends the token to React streaming state, producing a real-time typing effect.
20. On stream completion: the full assistant message (concatenated tokens) is persisted to the SQLite chat_messages table with role='bot', session_id, and UTC timestamp. If this is the first message, the session title is updated from the first 60 characters of the user query.

21. Ollama inference timeout (30 seconds): if no token is received within 30 seconds, FastAPI aborts the generator and sends an SSE error event. The frontend displays the timeout error message inline.

### 6.2.2 Role-Specific Checks in Query Flow

| Check | BA | FBA | QA |
|---|---|---|---|
| System prompt template applied | BRD Generation Prompt: focus on business goals, stakeholders, problem statement, scope, assumptions, constraints | FRD Generation Prompt: focus on functional requirements (shall statements), interfaces, data entities, validation rules | Test Pack Prompt: focus on test objectives, scope, preconditions, test steps, expected results, edge cases, negative scenarios |
| Qdrant retrieval filter | session_id filter only | session_id filter only | session_id filter + priority_tag boost (High = 1.3× score) |
| Role context tag prepended to response | [BA Context] | [FBA Context] | [QA Context] |
| Output format enforcement | Sections: Purpose, Business Background, Scope, Stakeholders, Requirements, Assumptions, Glossary | Sections: Introduction, Functional Requirements (shall), Interface Specs, Data Dictionary, Validation Rules | Table format: Test ID \| Test Scenario \| Preconditions \| Test Steps \| Expected Result \| Priority \| Status |
| Guardrail on off-role content | LLM instructed not to generate test cases or API specs | LLM instructed not to generate BRD-style stakeholder analysis | LLM instructed not to generate business requirement narratives |

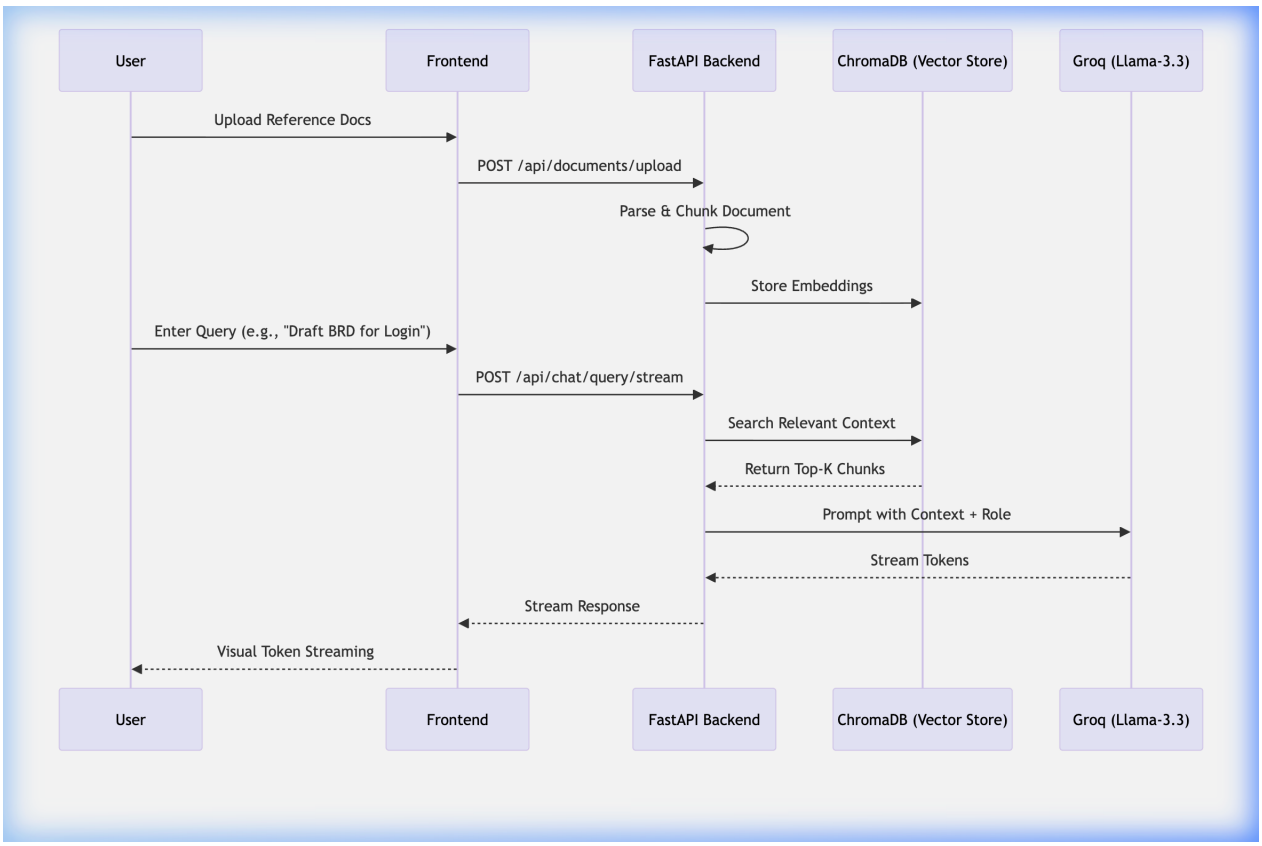


*Figure 3 — Data Flow Diagram: Document Ingestion & RAG Query Sequence*

# 7. High-Level Architecture

The SDLC Automation Copilot is built on a four-tier, fully on-premise, open-source architecture. Every component runs on local infrastructure with zero external cloud or API dependencies. The four tiers — Client, Application, Intelligence, and Data — communicate via well-defined HTTP/REST and gRPC interfaces, and are designed so that any individual component can be replaced without affecting adjacent tiers.

## 7.1 Tier 1 — Client Tier (Next.js 16 Frontend)

The Client Tier is the browser-based user interface built with Next.js 16 (React 19). It runs entirely in the user's browser and communicates with the Application Tier over HTTPS only. It is responsible for all UI rendering, state management, field-level validation, and real-time token streaming.
Core Responsibilities:

- Rendering all screens: Login, Role Selector, Document Upload Panel, Chat Window, and Session History Sidebar.
- Enforcing client-side field validations (character limits, file type and size checks, empty-state guards) before any server call is made — this reduces unnecessary server load and provides instant user feedback.
- Managing SSE streaming via the Fetch API ReadableStreamDefaultReader. Each SSE event (data: { token: '...' }) is decoded and appended to React state, producing a real-time typing effect without WebSocket dependency.
- No sensitive data is stored in browser localStorage or sessionStorage. The JWT session token is stored in an HTTP-only cookie managed by the Next.js server layer.
- The application is served by the Next.js standalone server on port 3000, behind the Nginx reverse proxy which handles TLS termination.

## 7.2 Tier 2 — Application Tier (FastAPI + RAG Service)

The Application Tier is the FastAPI backend hosted on Uvicorn (ASGI). It is the orchestration hub: receiving all requests from the Client Tier, coordinating with the Intelligence Tier for inference, and managing the Data Tier for persistence.
FastAPI Router Layer (Control Layer): Routes all HTTP requests. Validates inputs using Pydantic v2 models. Manages JWT authentication via a dependency injection guard on all protected endpoints. Exposes the following API surface:

- POST /api/auth/login — validates credentials, returns JWT.
- POST /api/chat/sessions — creates a new session for the authenticated user with selected role.
- GET /api/chat/sessions — returns all sessions for the authenticated user.
- GET /api/chat/sessions/{id}/messages — returns full message history for a session.
- DELETE /api/chat/sessions/{id} — soft-deletes the session (sets deleted_at timestamp).
- POST /api/documents/upload — multipart upload endpoint; triggers ingestion pipeline.
- POST /api/chat/query/stream — streaming RAG endpoint; returns text/event-stream SSE response.

RAG Service / Domain Layer (LangChain): Contains the core AI business logic. Orchestrates the full pipeline from query embedding through Qdrant retrieval, prompt assembly, Ollama LLM invocation, and SSE streaming. Implemented as a set of LangChain chains that abstract each sub-step, enabling component swaps (e.g. switching from Mistral-7B to LLaMA-3-8B) via configuration only.

The ASGI event loop (Uvicorn) handles all concurrent requests via cooperative coroutines — no additional threads are needed for I/O-bound operations such as LLM streaming or database queries. This enables at least 10 concurrent streaming sessions on a single Uvicorn process.

## 7.3 Tier 3 — Intelligence Tier (Ollama LLM + Embedding Model)

The Intelligence Tier hosts all AI inference components entirely on-premise. It has two sub-components, both of which execute without any external network calls:

Ollama LLM Server: Ollama is an open-source LLM runtime that manages model loading, GPU memory, and inference serving. It exposes a local REST API on port 11434. The FastAPI RAG Service sends POST http://localhost:11434/api/generate with the assembled prompt and stream: true. Ollama streams JSON token chunks back. The model (Mistral-7B or LLaMA-3-8B) is stored as GGUF or safetensor weights on the local filesystem and loaded into GPU VRAM on startup. Ollama is managed by systemd for automatic restart on failure.
Embedding Model (all-MiniLM-L6-v2): The HuggingFace sentence-transformers model is loaded directly into the FastAPI Python process using the sentence-transformers library. It is loaded once at backend startup and kept in memory for the lifetime of the process. No network call is made; all embedding computation is local CPU/GPU. The model converts both document chunks (during ingestion) and user queries (during RAG retrieval) into 384-dimensional dense vectors.
Why 384 dimensions? This embedding size provides an excellent trade-off: sufficient semantic resolution for document retrieval tasks, compact enough for fast Qdrant indexing, and small enough to keep memory and storage requirements low for a corpus of up to 50,000 chunks.

## 7.4 Tier 4 — Data Tier (Qdrant + SQLite + File Storage)

The Data Tier manages all persistent state. The FastAPI backend is fully stateless — all state is externalised to this tier, enabling clean restarts and future horizontal scaling.
Qdrant (Vector Store): Runs as an on-premise Docker container (qdrant/qdrant image). Stores all 384-dimensional chunk embeddings and associated metadata. Persists the vector index to the host filesystem via a Docker volume mount, ensuring no data loss on container restart. Qdrant uses HNSW (Hierarchical Navigable Small World) indexing for approximate nearest-neighbour search, providing sub-500 ms query response on 50,000+ vectors. It supports native metadata filtering (filter by session_id, role, priority_tag) executed in parallel with the vector search without performance degradation. Exposed on ports 6333 (REST) and 6334 (gRPC) on localhost only — not exposed externally.
SQLite (Relational Store): Stores user accounts (Users), chat sessions (Sessions), message history (Messages), document metadata (Documents), and user feedback (Feedback). Managed via SQLAlchemy ORM which provides a clean migration path to PostgreSQL in future updates with no application code changes. SQLite is chosen for because it requires zero infrastructure configuration and is sufficient for up to 50 concurrent users.
File Storage (Local FS / NAS): Uploaded source documents are stored on the local filesystem at a configured path (UPLOAD_DIR environment variable). The file path is recorded in the SQLite Documents table. For larger deployments, UPLOAD_DIR can point to a NAS mount point. No cloud object storage is used.

## 7.5 Cross-Tier Communication Summary

| Interaction | Source | Destination | Protocol / Method |
|---|---|---|---|
| User query submission | Client Tier (Browser) | Application Tier (FastAPI) | HTTPS POST (JSON body) |
| SSE token stream | Application Tier (FastAPI) | Client Tier (Browser) | HTTPS SSE (text/event-stream) |
| Document upload | Client Tier (Browser) | Application Tier (FastAPI) | HTTPS multipart/form-data POST |
| SSE status update (ingestion) | Application Tier (FastAPI) | Client Tier (Browser) | HTTPS SSE (text/event-stream) |
| Query embedding | Application Tier (RAG Service) | Intelligence Tier (MiniLM) | In-process Python call (no network) |

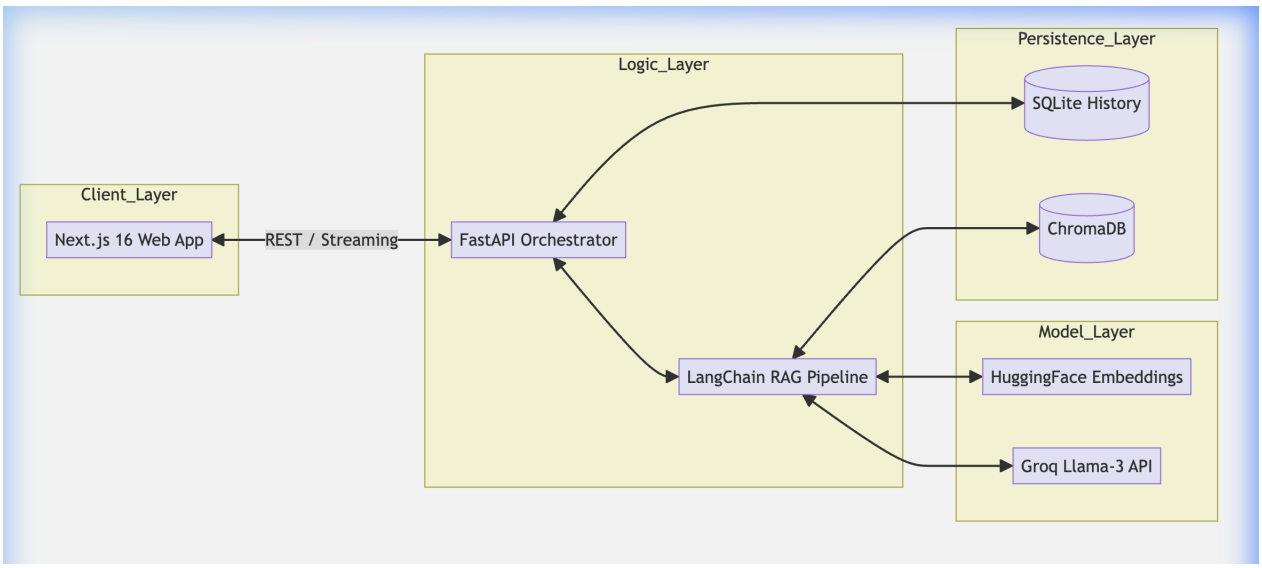


*Figure 4 — High-Level Architecture: Four-Tier On-Premise System*

| Interaction | Source | Destination | Protocol / Method |
|---|---|---|---|
| Chunk embedding (ingestion) | Application Tier (Ingestion Service) | Intelligence Tier (MiniLM) | In-process Python call (no network) |
| LLM inference request | Application Tier (RAG Service) | Intelligence Tier (Ollama) | HTTP POST localhost:11434/ api/generate |
| LLM token stream | Intelligence Tier (Ollama) | Application Tier (RAG Service) | HTTP streaming JSON (localhost) |
| Vector upsert (ingestion) | Application Tier (Ingestion Service) | Data Tier (Qdrant) | HTTP REST localhost:6333 |
| Vector similarity search | Application Tier (RAG Service) | Data Tier (Qdrant) | HTTP REST localhost:6333 |
| Session / message read-write | Application Tier (FastAPI) | Data Tier (SQLite) | SQLAlchemy ORM (file I/O) |
| File storage | Application Tier (Ingestion Service) | Data Tier (Local FS / NAS) | OS file write |

# 8. Component Architecture — Open-Source Selection, Rationale and Comparison

All components selected are open-source, on-premise deployable, and carry zero third-party API or cloud dependencies. This section documents each component, states why it was selected, and provides structured comparisons with alternatives.

## 8.1 LLM Inference — Ollama with Mistral-7B / LLaMA-3-8B

### Why Selected

- Fully on-premise: model weights stored locally; zero data transmitted to external services.
- Ollama provides a single-binary runtime with a simple, model-agnostic REST API, enabling model swapping (e.g., Mistral-7B to LLaMA-3-8B) via a single configuration change.
- Mistral-7B and LLaMA-3-8B are open-weight instruction-following models with strong performance on structured document generation tasks.
- GPU-accelerated via CUDA; CPU fallback supported for dev environments without GPU.
- Managed by systemd for automatic process supervision and restart.

### Component Comparison

| Option | On-Premise | Open-Source / Open-Weight | Cost | Performance | Third-Party Dependency | Decision |
|---|---|---|---|---|---|---|
| Ollama + Mistral-7B | ✓ | ✓ (Apache-2.0 / Mistral licence) | Free | High (GPU) | None | ✓ Selected |
| Ollama + LLaMA-3-8B | ✓ | ✓ (Meta LLaMA licence) | Free | High (GPU) | None | ✓ Alternate model option |
| vLLM (local) | ✓ | ✓ (Apache-2.0) | Free | Very High | None | ○ Higher ops complexity; Ollama preferred for ease of deployment |
| llama.cpp (local) | ✓ | ✓ (MIT) | Free | Medium (CPU) | None | ○ CPU-only fallback; lower throughput |
| Groq API (cloud) | ✗ | ✗ | Pay-per-token | Very High | Groq (cloud) | ✗ External API — eliminated |
| OpenAI API (cloud) | ✗ | ✗ | Pay-per-token | Very High | OpenAI (cloud) | ✗ External API — eliminated |

## 8.2 Vector Database — Qdrant

### Why Selected

- Open-source (Apache-2.0); self-hosted via a single Docker image (qdrant/qdrant).
- Production-grade HNSW indexing with sub-100 ms approximate nearest-neighbour search on 50,000+ vectors.
- Native payload filtering (filter by session_id, role, priority_tag) executed in parallel with vector search — no performance penalty.
- REST and gRPC APIs; official Python client (qdrant-client) available via pip.
- Persistent disk storage via Docker volume; no data loss on container restart.

- Qdrant Cloud also available if future cloud migration is needed, with identical API — no code changes required.

### Component Comparison

| Option | On-Premise | Open-Source | Metadata Filtering | Scale | Ease of Deployment | Decision |
|---|---|---|---|---|---|---|
| Qdrant | ✓ | ✓ (Apache-2.0) | ✓ Native payload filters | 50M+ vectors | Single Docker image | ✓ Selected |
| ChromaDB | ✓ | ✓ (Apache-2.0) | ○ Limited filtering | ~1M vectors | Single Python package | ○ Used in v1.0; replaced for production scale and filtering |
| Weaviate | ✓ | ✓ (BSD-3) | ✓ Native | Large scale | Docker (heavier) | ○ Viable alternative; higher resource footprint than |
| Milvus | ✓ | ✓ (Apache-2.0) | ✓ Native | Very large scale | Complex cluster setup | ○ Over-engineered for target corpus size |
| FAISS | ✓ | ✓ (MIT) | ✗ No metadata filtering | Very large | Library only; no server | ✗ Cannot filter by session_id or role without custom |
| Pinecone | ✗ | ✗ | ✓ | Very large | Managed cloud | ✗ Cloud-only; external dependency — eliminated |

## 8.3 Embedding Model — sentence-transformers/all-MiniLM-L6-v2

### Why Selected

- Fully local: model downloaded once (via pip install sentence-transformers); runs in-process with no network calls.
- 384-dimensional output: compact vectors balancing retrieval quality against storage and indexing cost.
- Top-tier performance on semantic similarity benchmarks (MTEB) relative to model size.
- Zero cost; no API key; no rate limits; no data leaves the server.

### Component Comparison

| Option | On-Premise | Dimensions | Semantic Quality | Cost | Third-Party Dependency | Decision |
|---|---|---|---|---|---|---|
| all-MiniLM-L6-v2 | ✓ | 384 | High (MTEB top-tier for | Free | None | ✓ Selected |

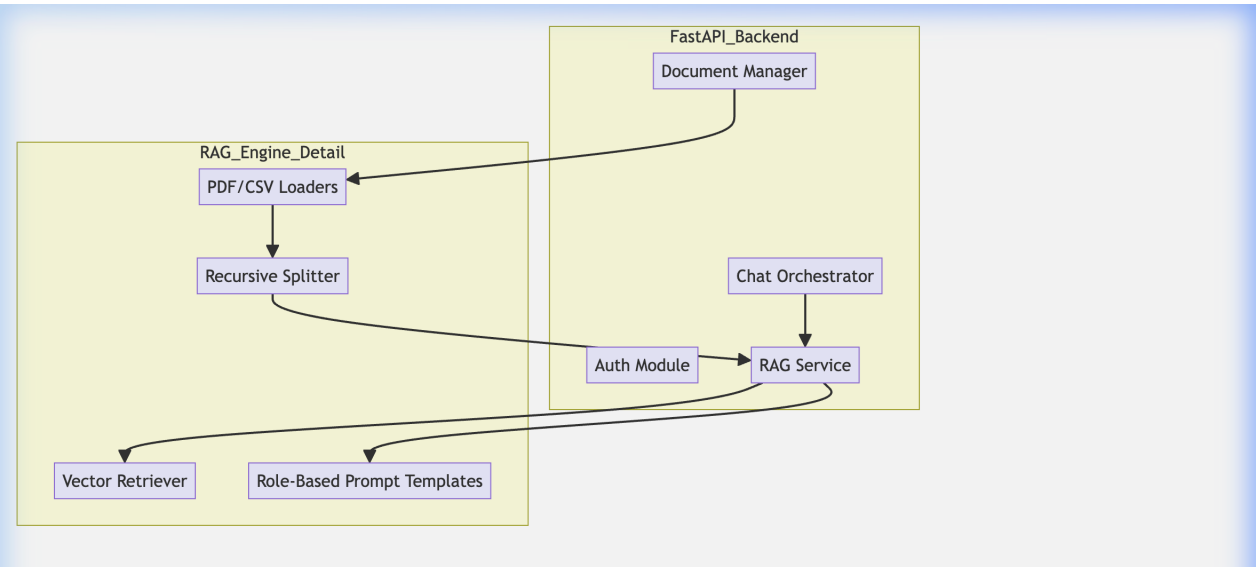


*Figure 5 — Component Architecture: FastAPI Backend & RAG Engine Detail*

| Option | On-Premise | Dimensions | Semantic Quality | Cost | Third-Party Dependency | Decision |
|---|---|---|---|---|---|---|
| all-mpnet-base-v2 | ✓ | 768 | Higher | Free | None | ○ 2× storage; marginal quality gain for this use-case |
| BGE-M3 (BAAI) | ✓ | 1024 | Very high; multilingual | Free | None | ○ Candidate for future updates multilingual support |
| E5-large (Microsoft) | ✓ | 1024 | Very high | Free | None | ○ Higher quality but 3× storage and 4× indexing time |
| OpenAI text-embedding-3 | ✗ | 1536 | Very high | Pay-per-token | OpenAI (cloud) | ✗ External API dependency — eliminated |
| Cohere Embed | ✗ | 1024 | High | Pay-per-token | Cohere (cloud) | ✗ External API dependency — eliminated |

## 8.4 Backend Framework — FastAPI

### Why Selected

- Native async/await (ASGI) is essential for non-blocking LLM streaming and concurrent document ingestion.
- Automatic OpenAPI/Swagger documentation generated from route and Pydantic schema definitions.
- Pydantic v2 integration provides fast, type-safe request/response validation with detailed error messages.
- StreamingResponse with media_type='text/event-stream' enables Server-Sent Events without additional libraries.

### Component Comparison

| Option | Async Native | Auto API Docs | Type Safety | SSE Support | Decision |
|---|---|---|---|---|---|
| FastAPI | ✓ (ASGI) | ✓ OpenAPI auto-generated | ✓ Pydantic v2 | ✓ StreamingResponse | ✓ Selected |
| Flask | ✗ (sync WSGI) | ✗ | ○ Marshmallow (optional) | ○ Requires flask-sse or custom | ✗ Synchronous model unsuitable for concurrent LLM streaming |
| Django | ✗ (primarily sync) | ✗ | ○ | ○ Channels required | ✗ Heavyweight; async support immature |
| Tornado | ✓ (async) | ✗ | ✗ | ✓ | ✗ No auto docs; higher boilerplate |

| Option | Async Native | Auto API Docs | Type Safety | SSE Support | Decision |
|---|---|---|---|---|---|
| Starlette | ✓ (ASGI) | ✗ | ○ | ✓ | ○ FastAPI is built on Starlette; FastAPI adds docs and validation |

## 8.5 RAG Orchestration — LangChain

### Why Selected

- Provider-agnostic abstraction: switching the LLM (Ollama Mistral-7B to LLaMA-3-8B), vector store (Qdrant to Weaviate), or embedding model requires only configuration changes.
- RecursiveCharacterTextSplitter provides semantically aware chunking with configurable chunk size and overlap.
- Built-in chain patterns (RetrievalQA, LLMChain) reduce boilerplate for retrieval and generation pipelines.
- Active open-source community; Apache-2.0 licensed.

### Component Comparison

| Option | Provider Agnostic | Built-in Chunking | Prompt Management | Maturity | Decision |
|---|---|---|---|---|---|
| LangChain | ✓ | ✓ RecursiveCharacterTextSplitter | ✓ PromptTemplate, ChatPromptTemplate | High | ✓ Selected |
| LlamaIndex | ✓ | ✓ | ✓ | High | ○ Stronger for document indexing pipelines; less flexible for custom chain logic |
| Haystack | ✓ | ✓ | ✓ | High | ○ Viable; more complex pipeline DSL |
| Custom Python | ✓ (manual) | Manual implementation | Manual | N/A | ✗ High ongoing maintenance cost; no community support |

## 8.6 Frontend Framework — Next.js 16 (React 19)

### Why Selected

- File-based routing reduces project boilerplate for the relatively contained UI surface area.
- React 19 server and client components improve rendering performance for the session sidebar and chat window.
- Native Fetch API ReadableStream support makes SSE consumption straightforward without WebSocket.

- Lucide-React (MIT licensed) provides a clean, consistent open-source icon set.

## 8.7 Model Training — Hugging Face PEFT (LoRA) + MLflow

### Why Selected

- PEFT LoRA: updates less than 1% of model parameters, making fine-tuning feasible on a single consumer GPU (e.g. RTX 3090 or A100). No full-weight training infrastructure required.
- MLflow: open-source (Apache-2.0) experiment tracking, artefact storage, and model registry. Self-hosted; zero external dependency.
- Hugging Face Trainer: open-source, well-documented training loop with built-in gradient accumulation, evaluation, and checkpoint management.

### Component Comparison

| Option | On-Premise | Open-Source | GPU Efficient | Experiment Tracking | Decision |
|---|---|---|---|---|---|
| PEFT LoRA + MLflow | ✓ | ✓ | ✓ <1% params updated | ✓ MLflow | ✓ Selected |
| Full fine-tuning (Trainer) | ✓ | ✓ | ✗ Full GPU cluster needed | ✓ MLflow | ✗ Infrastructure requirements too high for internal team |
| OpenAI fine-tuning API | ✗ | ✗ | N/A | ✗ | ✗ External API; data leaves on-premise environment |
| Axolotl | ✓ | ✓ | ✓ | ○ | ○ Higher ops complexity; PEFT preferred for simplicity |

# 9. Model Training and Continuous Improvement Flow

This section defines the full pipeline for training, fine-tuning, evaluating, and deploying updated versions of the on-premise LLM over time. All components are open-source and run entirely on the local server.

## 9.1 Training Strategy Overview

| Phase | Method | Trigger | Goal |
|---|---|---|---|
| Phase 1: Base Model Serving | Ollama serving pre-trained Mistral-7B or LLaMA-3-8B weights — no training | At initial deployment | Functional baseline; zero training overhead |
| Phase 2: LoRA Fine-Tuning | PEFT LoRA adapters trained on curated SDLC Q&A pairs from user feedback | Monthly or when ≥ 200 quality feedback pairs accumulate | Improve BRD/FRD/Test Pack generation quality; align tone and format to organisational standards |
| Phase 3: Continuous Improvement | Incremental LoRA adapter updates using new feedback; adapters merged and redeployed | Quarterly or when ≥ 500 new feedback pairs accumulated since last run | Sustained alignment improvement; model increasingly tailored to organisational language and conventions |

## 9.2 Step 1 — Feedback Collection

Every assistant response displays a Thumbs Up / Thumbs Down feedback widget in the Chat Window. Users may optionally provide a correction note (maximum 500 characters) explaining what was wrong or missing in the response.
Feedback is stored in the SQLite Feedback table with the following fields:

| Field | Type | Description |
|---|---|---|
| id | Integer PK | Unique feedback identifier |
| message_id | Integer FK → Messages | The assistant message being rated |
| session_id | String (UUID) FK → Sessions | Parent session |
| rating | Integer (1 or -1) | 1 = thumbs-up (positive); -1 = thumbs-down (negative) |
| correction_note | Text (nullable) | Optional user correction; maximum 500 characters |
| created_at | DateTime | UTC timestamp of feedback submission |

- Positive ratings (thumbs-up) contribute input → output training pairs.
- Negative ratings with correction notes contribute input → corrected_output training pairs.
- Negative ratings without correction notes are flagged for review but not automatically added to the training set.

## 9.3 Step 2 — Dataset Curation

- Admin script: scripts/training/curate_dataset.py exports feedback records from SQLite into a JSONL fine-tuning dataset.

- Format per record: { "instruction": "<user_query>", "context": "<retrieved_chunks>", "response": "<approved_response>" }.
- Curation thresholds: minimum 200 high-quality pairs (thumbs-up rating) required before a fine-tuning run is triggered.
- Curated datasets are versioned (dataset_v1.jsonl, dataset_v2.jsonl) and stored in the local datasets/ directory, tracked as MLflow artefacts.
- A held-out evaluation set of 50 manually curated Q&A pairs (not used in training) is maintained in datasets/eval_set.jsonl for consistent benchmark evaluation across runs.

## 9.4 Step 3 — LoRA Fine-Tuning

- Fine-tuning script: scripts/training/finetune_lora.py — wraps Hugging Face Trainer with PEFT LoRA configuration and MLflow logging.
- LoRA configuration: rank r=8, alpha=16, dropout=0.05, target_modules=['q_proj', 'v_proj']. Updates less than 1% of total model parameters.
- Training hyperparameters (baseline): learning_rate=2e-4, num_train_epochs=3, per_device_train_batch_size=4, gradient_accumulation_steps=4.
- MLflow run is created at the start of each fine-tuning job and logs all hyperparameters, dataset version, per-step training loss, and artefact paths.
- LoRA adapter checkpoint is saved to adapters/run_{timestamp}/ and logged as an MLflow artefact.

## 9.5 Step 4 — Evaluation

- Evaluation script: scripts/training/evaluate.py runs the new adapter against the 50-item held-out eval set.
- Metrics computed and logged to MLflow:
  - ROUGE-L: measures structural alignment between generated and reference output.
  - BLEU-4: measures n-gram text overlap between generated and reference output.
  - LLM-as-Judge Score: the base model (Mistral-7B without adapter) rates each generated response vs. the reference on a 1–5 scale for accuracy, completeness, and format compliance.
- Acceptance gate: a new adapter is accepted only if all three metrics equal or exceed the scores from the previous deployed adapter. Regression on any metric causes automatic rejection and the previous adapter remains active.
- MLflow comparison view is used by the architecture team to review metric changes across runs before approving promotion.

## 9.6 Step 5 — Adapter Deployment

- Approved LoRA adapters are merged into the base model weights using PEFT's merge_and_unload() function, producing a full merged-weight model.
- Merged weights are copied to the Ollama models directory and registered as a new named model version (e.g. sdlc-copilot-v2).
- The application configuration (OLLAMA_MODEL environment variable) is updated to point to the new model version.
- Ollama is restarted via systemctl restart ollama. Downtime during restart is typically under 30 seconds.
- The active model version is displayed in the UI footer (e.g. 'Model: sdlc-copilot-v2') for operational transparency.
- Blue-green model deployment (zero-downtime model updates) is planned for future updates.
- Rollback: if post-deployment issues are observed, the OLLAMA_MODEL variable is reverted to the previous version name and Ollama is restarted — full rollback within 2 minutes.

## 9.7 MLflow Experiment Tracking — Logged Artefacts

| Artefact | Detail |
|---|---|
| Dataset version | JSONL filename, SHA-256 hash, and record count logged per run |
| Hyperparameters | Learning rate, LoRA rank, alpha, dropout, epochs, batch size, gradient accumulation |
| Training loss curve | Per-step loss logged; visualised in MLflow UI |
| Evaluation metrics | ROUGE-L, BLEU-4, and LLM-judge score logged per epoch and at final evaluation |
| LoRA adapter checkpoint | Full adapter directory logged as MLflow artefact; accessible for rollback |
| Model lineage | Base model name + adapter version linked in MLflow run metadata |
| Acceptance decision | Accept / Reject + reason logged as a run tag |

# 10. Logical View

## 10.1 N-Tier Layer Overview

| Layer | Tier | Components | Technology |
|---|---|---|---|
| Presentation | Client | ChatWindow, SidebarSessionList, RoleSelector, DocumentUploadPanel, StreamingMessageBubble, FeedbackWidget | Next.js 16, React 19, Lucide-React |
| Control | Application | FastAPI Routers: auth_router, sessions_router, query_router, documents_router; Uvicorn ASGI server; JWT middleware | FastAPI, Uvicorn, PyJWT, Pydantic v2 |
| Domain | Application | RAG Service (LangChain RetrievalQA chain), Document Ingestion Service, Session Management Service, Prompt Template Selector | LangChain, PyPDF2, python-docx, Pandas |
| Intelligence | Intelligence | LLM Inference (Ollama / Mistral-7B or LLaMA-3-8B), Embedding Model (all-MiniLM-L6-v2) | Ollama (local), sentence-transformers (local) |
| Resource | Data | Qdrant Client, SQLAlchemy Session Factory, File Storage Manager, ClamAV Client | qdrant-client, SQLAlchemy, clamd |
| Common | Cross-cutting | Pydantic Schemas, Configuration Module (.env), Document Parser Utilities, Logging Module | python-dotenv, structlog |

## 10.2 Primary Functional Areas

- Document Ingestion Pipeline: Upload → ClamAV scan → Parser (PyPDF2 / python-docx / Pandas) → RecursiveCharacterTextSplitter → all-MiniLM-L6-v2 embedding → Qdrant upsert → SQLite Documents record → SSE status event.
- RAG Query and Generation: Query → all-MiniLM-L6-v2 embedding → Qdrant cosine similarity search (role-filtered + priority-boosted for QA) → chunk retrieval → role prompt assembly → Ollama LLM streaming → SSE token stream → SQLite message persistence.
- Session Management: POST /api/chat/sessions creates session → GET /api/chat/sessions lists sessions → GET /api/chat/sessions/{id}/messages retrieves history → DELETE soft-deletes → session list sidebar rendered in real time.

## 10.3 Use-Case Realizations

### Document Generation Package (RAG Core)

22. User enters query and role is confirmed from session context.
23. Browser UI sends POST /api/chat/query/stream (query, session_id, role).
24. FastAPI validates JWT and session ownership.
25. RAG Service encodes query: embedding = MiniLM.encode(query).

26. Qdrant returns top_k=5 chunks (cosine similarity, filtered by session_id; QA priority boost applied).
27. assemble_prompt(chunks, role_template, query) constructs fused prompt.
28. Ollama REST API called with fused prompt and stream=true.
29. FastAPI async generator yields SSE events (data: { token: '...' }) to browser.
30. Browser renders tokens progressively. On completion, full message persisted to SQLite.

# 11. Process View

The SDLC Automation Copilot has one primary process: the asynchronous request-response cycle managed by FastAPI on the Uvicorn ASGI server. The ASGI model handles concurrent requests by scheduling coroutines on a single-threaded event loop, making threads unnecessary for I/O-bound operations (Ollama calls, Qdrant queries, SQLite reads/writes).

## 11.1 Session State Machine

The session lifecycle state machine governs how a user session transitions through operational states from creation through document generation:

| Transition | Trigger | Description |
|---|---|---|
| (Initial) → Session Created | User clicks 'Start Session' after role selection | Session record created in SQLite; session_id and role stored |
| Session Created → Idle | Session creation confirmed | Default state; no documents uploaded yet; chat disabled |
| Idle → IngestionPending | User clicks 'Upload & Process' | Files accepted; client-side validation passed; upload in progress |
| IngestionPending → Processing | FastAPI receives upload; ClamAV scan passed | Ingestion pipeline started; chunking and embedding in progress |
| Processing → Ready | All chunks embedded and persisted to Qdrant | Session ready for queries; send button enabled; status badges show 'Ready' |
| Ready → Generating | User submits a query | RAG retrieval and Ollama streaming begun; send button disabled |
| Generating → Ready | SSE stream completes or user clicks Stop | Message persisted; session returns to Ready state for next query |
| Any State → Idle | User clicks 'Purge Context' or logs out | Session context cleared; documents remain in Qdrant but session reset |

## 11.2 Concurrency Model

- Multiple concurrent streaming sessions are served by the same Uvicorn event loop via cooperative coroutines.
- No additional worker processes or threads are required for up to 10 concurrent users.
- For heavier load (10+ concurrent users), Uvicorn can be scaled horizontally behind Nginx with multiple worker processes (--workers N flag).
- Document ingestion is executed as a FastAPI BackgroundTask — it does not block the HTTP response.

# 12. Deployment View

The SDLC Automation Copilot is deployed as a fully on-premise single-server application. All services run on one Linux host. The Nginx reverse proxy handles TLS termination and routes traffic between the frontend and backend.

## 12.1 On-Premise Deployment Topology

| Node | Component | Technology | Port / Access |
|---|---|---|---|
| Client Machine | Browser | Chrome / Firefox / Edge | HTTPS 443 (outbound) |
| App Server | Reverse Proxy | Nginx (TLS termination, /api/* routing) | 80 (redirect to 443) / 443 |
| App Server | Frontend | Next.js 16 (Node.js standalone) | 3000 (internal, proxied by Nginx) |
| App Server | Backend API | FastAPI + Uvicorn (ASGI) | 8000 (internal, proxied by Nginx) |
| App Server | LLM Inference | Ollama (Mistral-7B / LLaMA-3-8B) | 11434 (localhost only) |
| App Server | Vector Store | Qdrant (Docker container) | 6333 REST, 6334 gRPC (localhost only) |
| App Server | Relational DB | SQLite (file on disk) | Internal (file I/O via SQLAlchemy) |
| App Server | Document Store | Local filesystem / NAS mount | Internal (file path from env var) |
| App Server | Virus Scanner | ClamAV (clamd daemon) | Internal Unix socket |
| App Server | Training UI | MLflow server | 5000 (localhost only, admin access) |

## 12.2 Future Update Roadmap — Containerisation and Scale-Out

- Docker Compose for local development: all services defined in docker-compose.yml.
- Kubernetes for production: FastAPI pods behind HPA; Qdrant cluster deployment; PostgreSQL migration.
- Nginx replaced by ingress controller for Kubernetes deployments.
- Blue-green model deployment for zero-downtime Ollama model updates.

# 13. Implementation View

## 13.1 Monorepo Structure

The codebase is structured as a monorepo with two top-level directories:

- /frontend — Next.js 16 (React 19) application: pages, components, hooks, API client.
- /backend — FastAPI application: routers, services, models, schemas, config, training scripts.

## 13.2 Key Layers and Modules

| Layer | Directory / Module | Description |
|---|---|---|
| Presentation | frontend/components/ ChatWindow | Message history rendering with markdown support and syntax highlighting |
| Presentation | frontend/components/ SidebarSessionList | Session list with search, rename, delete, and role badges |
| Presentation | frontend/components/ RoleSelector | Role card selection modal; Start Session button |
| Presentation | frontend/components/ DocumentUploadPanel | Drop zone, file list, status badges, priority tag dropdown (QA) |
| Presentation | frontend/components/ StreamingMessageBubble | Live SSE token rendering with typing cursor and feedback widget |
| Control | backend/routers/auth_router.py | POST /api/auth/login — credential validation; JWT issuance |
| Control | backend/routers/ sessions_router.py | CRUD for chat sessions; GET /sessions, POST /sessions, DELETE /sessions/{id} |
| Control | backend/routers/query_router.py | POST /api/chat/query/stream — SSE streaming RAG endpoint |
| Control | backend/routers/ documents_router.py | POST /api/documents/upload — multipart upload; triggers ingestion background task |
| Domain | backend/services/rag_service.py | LangChain RAG pipeline: embed query, Qdrant retrieval, prompt assembly, Ollama call |
| Domain | backend/services/ ingestion_service.py | Parse (PyPDF2/python-docx/Pandas), chunk (RecursiveCharacterTextSplitter), embed (MiniLM), upsert (Qdrant) |
| Domain | backend/services/ session_service.py | Session and message CRUD; SQLAlchemy session factory |
| Domain | backend/prompts/ role_templates.py | BA, FBA, and QA system prompt templates; prompt assembly function |
| Intelligence | backend/models/ embedding_model.py | Singleton loader for all-MiniLM-L6-v2; encode() wrapper |
| Resource | backend/db/qdrant_client.py | Qdrant collection management, upsert, and cosine search with payload filter |
| Resource | backend/db/sqlite_models.py | SQLAlchemy ORM models: User, Session, Message, Document, Feedback |

| Layer | Directory / Module | Description |
|---|---|---|
| Resource | backend/db/file_storage.py | File write, path management, and ClamAV scan invocation |
| Common | backend/schemas/ | Pydantic v2 request/response schemas shared across routers and services |
| Common | backend/config.py | Centralised settings: OLLAMA_MODEL, QDRANT_HOST, UPLOAD_DIR, CHUNK_SIZE loaded from .env |
| Training | backend/scripts/training/curate_dataset.py | Exports feedback records to JSONL training dataset |
| Training | backend/scripts/training/finetune_lora.py | PEFT LoRA fine-tuning script with MLflow logging |
| Training | backend/scripts/training/evaluate.py | ROUGE-L, BLEU-4, LLM-judge evaluation against held-out eval set |

# 14. Data View

Persistent state is managed across two stores: a relational database (SQLite via SQLAlchemy ORM) and a vector database (Qdrant). A new Feedback table is added in to support the model training pipeline.

## 14.1 Relational Database — SQLite

### Table: Users

| Column | Type | Constraint | Description |
|---|---|---|---|
| id | String (UUID) | PK | Unique user identifier |
| username | String | UNIQUE, NOT NULL | Login username (3–50 alphanumeric + underscore) |
| hashed_password | String | NOT NULL | bcrypt-hashed password |
| role | String | NOT NULL | BA / FBA / QA — default role for new sessions |
| created_at | DateTime | NOT NULL | UTC timestamp of account creation |

### Table: Sessions (chat_sessions)

| Column | Type | Constraint | Description |
|---|---|---|---|
| id | String (UUID) | PK | Unique session identifier |
| user_id | String (UUID) | FK → Users, NOT NULL | Owner of the session |
| role | String | NOT NULL | Role active for this session (BA / FBA / QA) |
| title | String (80) | NOT NULL | Derived from first 60 chars of first query; user-editable |
| created_at | DateTime | NOT NULL | UTC timestamp of session creation |
| deleted_at | DateTime | Nullable | Set on soft delete; NULL = active session |

### Table: Messages (chat_messages)

| Column | Type | Constraint | Description |
|---|---|---|---|
| id | Integer | PK, Auto-increment | Unique message identifier |
| session_id | String (UUID) | FK → Sessions, NOT NULL | Parent session |
| role | String | NOT NULL | 'user' or 'bot' — message sender |
| content | Text | NOT NULL | Full text of the message (user query or assistant response) |
| timestamp | DateTime | NOT NULL | UTC timestamp of message creation |

### Table: Documents

| Column | Type | Constraint | Description |
|---|---|---|---|
| id | Integer | PK, Auto-increment | Unique document identifier |
| session_id | String (UUID) | FK → Sessions, NOT NULL | Session that ingested this document |
| filename | String (200) | NOT NULL | Original uploaded filename |
| path | String | NOT NULL | Filesystem path to the stored file |
| size_bytes | Integer | NOT NULL | File size in bytes |
| status | String | NOT NULL | Pending / Processing / Ready / Error |
| priority_tag | String | Nullable | High / Medium / Low — QA role only; NULL for BA and FBA |
| created_at | DateTime | NOT NULL | UTC timestamp of document upload |

Table: Feedback (new in )

| Column | Type | Constraint | Description |
|---|---|---|---|
| id | Integer | PK, Auto-increment | Unique feedback identifier |
| message_id | Integer | FK → Messages, NOT NULL | The assistant message being rated |
| session_id | String (UUID) | FK → Sessions, NOT NULL | Parent session |
| rating | Integer | NOT NULL | 1 = thumbs-up; -1 = thumbs-down |
| correction_note | Text | Nullable | Optional user correction; max 500 characters |
| created_at | DateTime | NOT NULL | UTC timestamp of feedback submission |

## 14.2 Vector Database — Qdrant Collection Schema

| Field | Type | Description |
|---|---|---|
| id | String (UUID) | Unique identifier for each document chunk |
| embedding | float[384] | 384-dimensional dense vector from all-MiniLM-L6-v2 |
| payload.document | Text | Raw text content of the chunk |
| payload.source | String | Original filename of the uploaded document |
| payload.chunk_index | Integer | Position of this chunk within the source document |
| payload.session_id | String (UUID) | Session that ingested this chunk — used as primary retrieval filter |
| payload.role | String | BA / FBA / QA — role active at ingestion time |
| payload.priority_tag | String | High / Medium / Low — populated by QA role; defaults to 'Medium' |

# 15. Size and Performance

## 15.1 Volume Estimates

| Metric | Estimate |
|---|---|
| Registered users (internal) | 20–50 BA, FBA, and QA team members |
| Document generation sessions per day | 50 sessions per day; peak during sprint planning and release cycles |
| Documents ingested into Qdrant per deployment | 200–500 documents generating approximately 10,000–50,000 embedded chunks |
| Chat messages stored in SQLite per day | 500–2,000 messages per day across all active sessions |
| Feedback records per month | Estimated 200–500 feedback records per month at steady state |
| LoRA fine-tuning dataset (per training run) | 200–2,000 curated JSONL records |

## 15.2 Performance Targets

| Metric | Target | Notes |
|---|---|---|
| Time-To-First-Token (TTFT) | < 3 seconds | GPU-accelerated Ollama; CPU fallback approximately 10–30 seconds |
| Qdrant cosine similarity search (top-k=5) | < 500 ms | For corpus up to 50,000 chunks with HNSW index |
| Full PDF ingestion (20-page) | < 60 seconds | Parse + chunk + embed + Qdrant upsert |
| Concurrent streaming sessions (no degradation) | ≥ 10 simultaneous users | Single Uvicorn process; scale out via --workers N for higher load |
| UI page load time | < 2 seconds | Local network; Next.js standalone server + Nginx |
| Login response time | < 500 ms | bcrypt hash comparison + JWT issuance |
| Session history list load | < 1 second | SQLite query on indexed user_id column |

# 16. Quality Attributes

| Quality Attribute | Description | Architectural Solution |
|---|---|---|
| Scalability | Handle increasing concurrent requests and growing document corpus without degradation | FastAPI ASGI async model supports high concurrency on a single process. Qdrant can be replaced with Qdrant cluster or Weaviate for very large corpora. Stateless API design enables horizontal scaling behind Nginx load balancer. |
| Reliability / Availability | System restarts cleanly and resumes without data loss after failure | Stateless FastAPI backend; all state externalised to SQLite and Qdrant. Ollama supervised by systemd (auto-restart). Retry logic with exponential backoff on Ollama inference failures. |
| Portability | Deployable on developer laptops, on-premise servers, or future cloud VMs with minimal config change | Single .env file controls all environment-specific settings. Docker Compose planned for future updates. No external API keys; no cloud lock-in. |
| Security | Protect proprietary SDLC documents and session history from unauthorised access | JWT authentication (RS256). Role-based prompt context injection. HTTPS/TLS via Nginx. ClamAV file scanning. Immutable audit trail in SQLite. No external data transmission. |
| Maintainability | Modify, extend, or replace individual components without impacting adjacent layers | N-tier layer separation. LangChain abstraction enables LLM swaps (Mistral-7B → LLaMA-3-8B) via config change. SQLAlchemy ORM enables SQLite → PostgreSQL migration with no application code changes. |
| Extensibility | Support planned future capabilities: Jira sync, Word/ PDF export, admin analytics, additional roles | FastAPI router structure allows new endpoint groups (/api/export, /api/jira) without modifying existing routes. Role prompt template architecture supports new roles without structural changes. LangChain supports new LLM providers via provider-specific integrations. |
| Zero Third-Party Dependency | No external APIs, cloud services, or paid components | Ollama (local LLM), Qdrant (local vector DB), all-MiniLM-L6-v2 (local embedding), ClamAV (local virus scan), MLflow (local experiment tracking). All components open-source and self-hosted. |

*End of Document — Software Architecture Document*